A pan-tracking camera on the Arduino UNO Q

Heterogeneous inference of YOLOv8n and LightFC on the QRB2210 CPU (4 Arm Cortex-A53) and GPU (Adreno 702), and using computer vision to drive a motor using the Arm Cortex-M33

Bence Danko · ACM/IEEE SEC 2026 Student Innovation Challenge · Minimalist Edge AI, Hardware Track

Problem, and why on the device

Select a class, e.g. "person", and the camera locates and uses a motor to pan to center.

Requirement	Reason
Privacy	Video stays on the board.
Latency	The motor loop runs on every frame. A network round trip would add to each step.
No network	Field sites often have none.

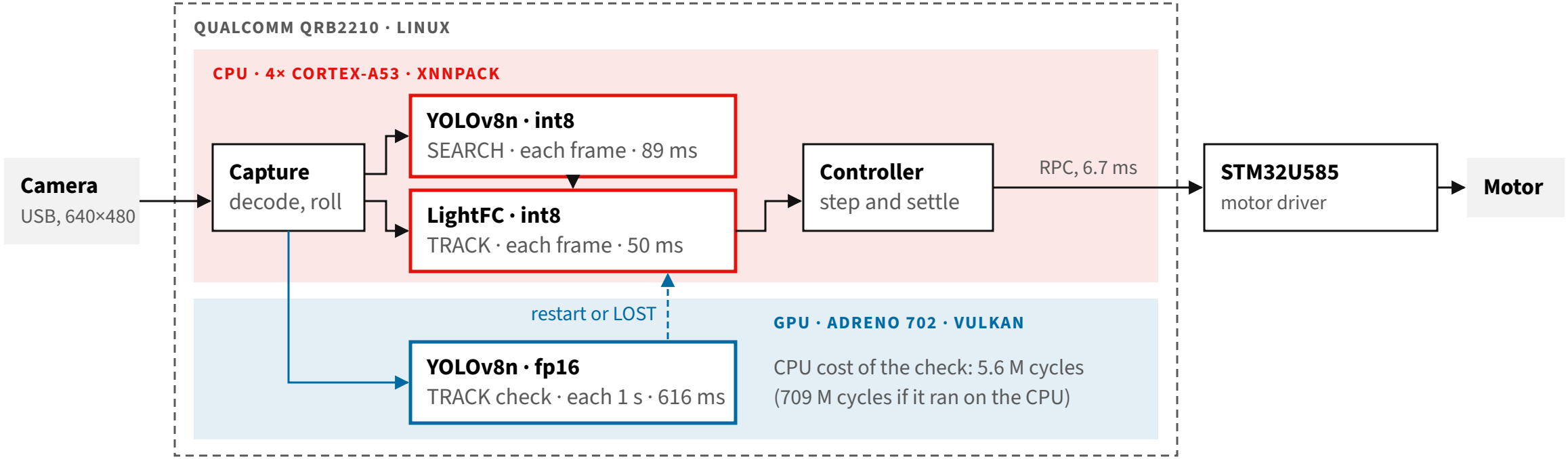
Uses: laboratories, lecture rooms, care rooms, field and wildlife observation.

Hardware

Unit	Specification	Effect
CPU	4× Cortex-A53, 2.0 GHz, Armv8.0	NEON SIMD; no fp16 arithmetic
GPU	Adreno 702, Vulkan 1.0	fp32 and fp16; no int8 (no 8-bit storage)
NPU	None usable	No QNN
RAM	3.6 GB	
MCU	STM32U585	Drives the motor from the tracker's commands
Power sensor	None	No energy measurement

Runtime: ExecuTorch 1.5.1, XNNPACK on the CPU, Vulkan on the GPU. No PyTorch on the board.

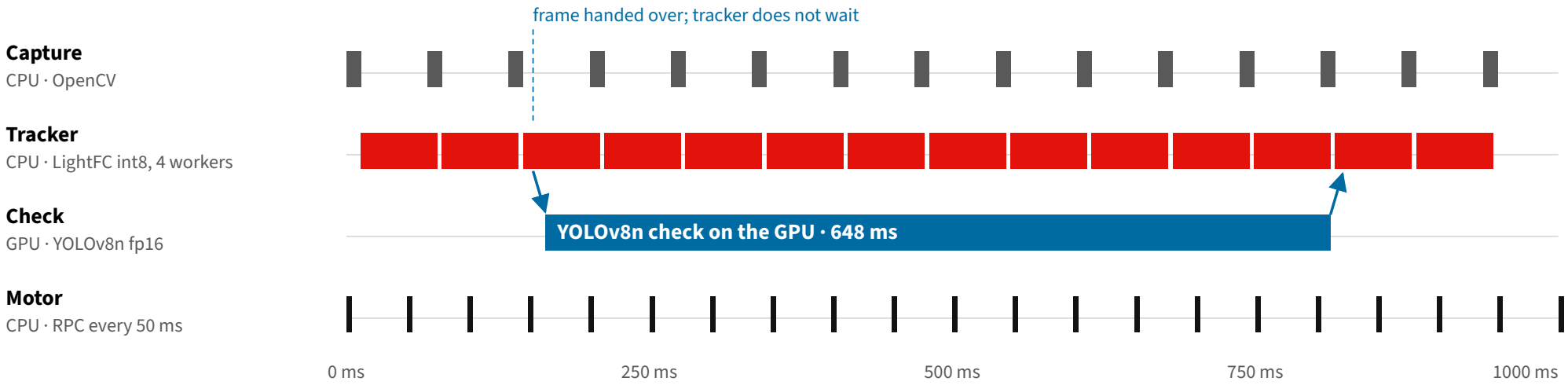
What runs where



Per-frame work on the CPU. The once-a-second check on the GPU, which is otherwise idle.

Software pipeline

One process, five threads. CPU and GPU run concurrently.



- Inference and image calls run in C++ (`libinfer.so`, OpenCV) and release the Python GIL.
- One-frame slot: newest frame only, no queue.
- The tracker hands a frame to the GPU check and does not wait for the result.
- Live: GPU 60 % busy while the process uses 317 % of 400 % CPU.

Software stack

Unit	Software
CPU, 4× Cortex-A53	ExecuTorch 1.5.1 + XNNPACK (NEON kernels, 4 threads), in our `libinfer.so`: 5-function C API, called via `ctypes`.
GPU, Adreno 702	ExecuTorch Vulkan backend → Mesa Turnip 25.2.6 → msm kernel driver, Linux 6.16.7
MCU, STM32U585	Arduino sketch, Zephyr core. RPCs `set_motor`, `stop` via Arduino_RouterBridge
Linux → MCU link	msgpack-rpc → `/run/arduino-router.sock` → arduino-router → STM32
Application	Python 3.13; OpenCV 5.0 (capture, decode, crop); NumPy 2.5 (box decode, NMS); Flask 3.1 (UI)
Export (laptop)	PyTorch 2.14, ExecuTorch 1.5.1 exporter; int8 via torchao PT2E + XNNPACKQuantizer; fp16 via `force_fp16`

ExecuTorch for CPU or GPU delegation

Export, on the laptop

```
if variant == "xnn8":                  # int8 for the CPU
    model = quantize_pt2e(model, example, calib)
part = {
    "xnn8": XnnpackPartitioner(),      # CPU
    "vk16": VulkanPartitioner(
        {"force_fp16": True}),         # GPU
}[variant]
ep = torch.export.export(model, example)
prog = to_edge_transform_and_lower(
    ep, partitioner=[part]).to_executorch()
prog.write_to_file(open(out_path, "wb"))
```

Run, on the board

```
cpu = Model("yolov8n_xnn8_320.pte", threads=4)
gpu = Model("yolov8n_vk16_320.pte")

boxes = cpu.run(image)   # same call
boxes = gpu.run(image)   # same call
```

`Model`: our `ctypes` wrapper on `libinfer.so`. The delegate (XNNPACK or Vulkan) is inside the `.pte`.

Models

	YOLOv8n	LightFC (MobileNetV2)
Task	Find an object of a COCO class	Follow it
Classes	80 (COCO)	Any object (template)
Input	320×320 image	128×128 template, 256×256 search area
Runs	SEARCH: every frame. TRACK: check once a second	TRACK: every frame
Used as	CPU int8 for SEARCH, GPU fp16 for the check	CPU int8

State	Operation
SEARCH	YOLOv8n on every frame; motor still
TRACK	LightFC on every frame; motor pulses; YOLOv8n check once a second
LOST	Motor stops; back to SEARCH

Test data and calibration

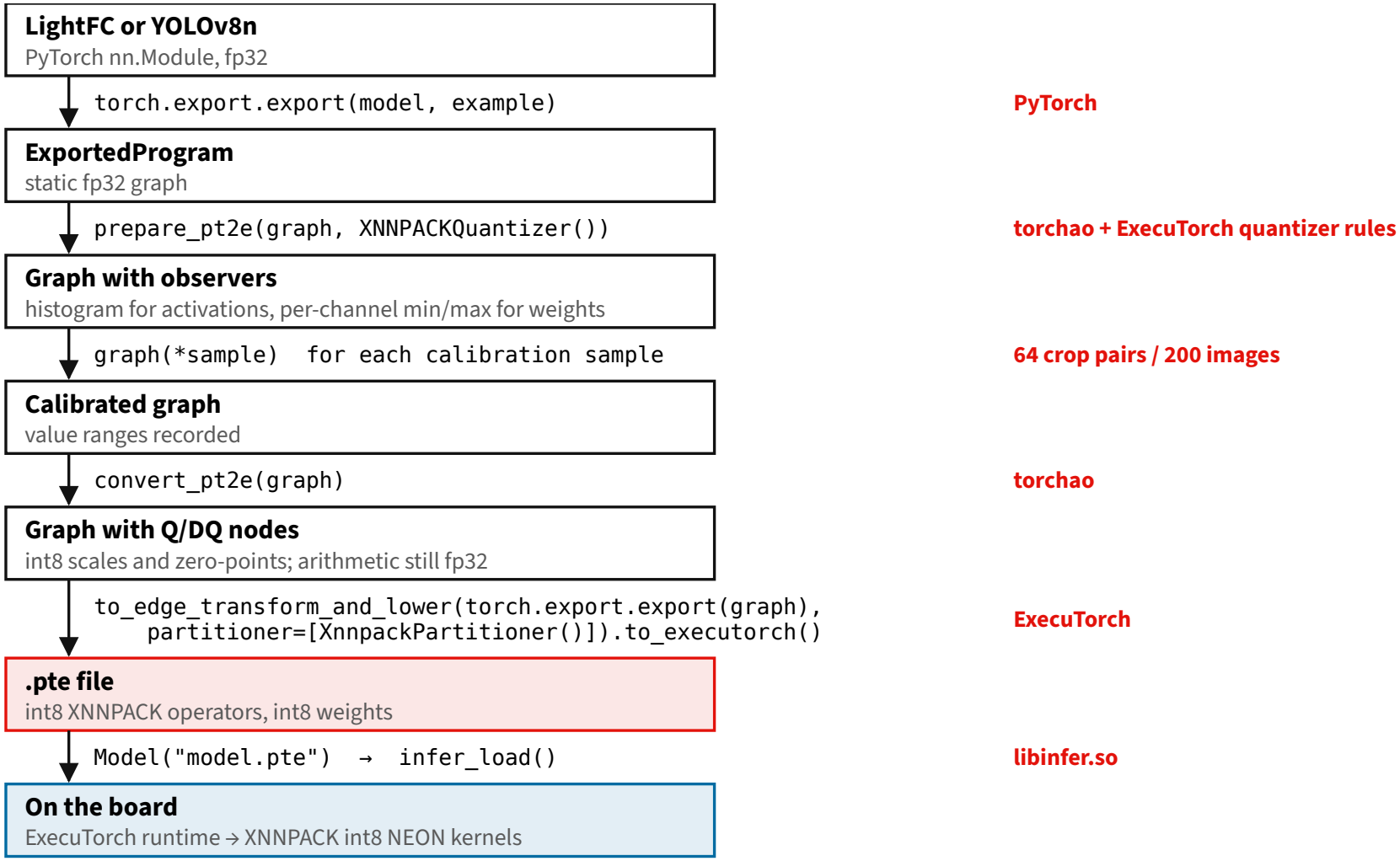
	LightFC	YOLOv8n
Weights	Authors' release (LaSOT, GOT-10k, COCO, TrackingNet). Not retrained.	Ultralytics release (COCO). Not retrained.
int8 calibration	64 template/search crop pairs from 6 OTB100 sequences outside the test set: Bolt, Car4, Coke, Crossing, Deer, Dog1	200 images (14 sequences)
Accuracy test	OTB100, 8 sequences: Basketball, Biker, BlurCar2, David3, Girl2, Human3, Jogging-1, Walking2. 5,709 frames. Success AUC, precision at 20 px	No mAP. Box agreement only.
Agreement test	20 crop pairs, score difference vs PyTorch	Host: vs Ultralytics (OTB100 first frames, 10 camera frames). Board: vs CPU fp32 (first frame of each test sequence)

OTB100: Wu, Lim and Yang, TPAMI 2015. Sequences chosen for horizontal motion, fast motion, occlusion and scale change. All accuracy tests run on the board.

Quantization

	LightFC	YOLOv8n
int8 on the CPU, XNNPACK		
Method	Static post-training quantization, PyTorch PT2E + XNNPACKQuantizer	
Weights	int8, symmetric, per output channel; range −127…127, zero-point 0, scale = max |w| / 127	
Activations	int8, asymmetric, per tensor; range −128…127; scale and zero-point from a histogram observer	
Quantized part	Per block; fusion and head calibrated on fp32 outputs of the preceding blocks	Whole network to the raw outputs; box decode stays fp32
Calibration data	64 crop pairs	200 images
Kernels	XNNPACK int8 NEON kernels	
Result vs fp32	**−43 %** latency, **−73 %** size; AUC 0.625 vs 0.581	**−20 %** latency, **−74 %** size; box IoU 0.972
fp16 on the GPU, Vulkan		
Method	Cast at export (`force_fp16`); weights and activations fp16; no calibration	
Result vs fp32	−11 % latency, −50 % size; AUC unchanged (0.581)	−22 % latency, −50 % size; box IoU 0.999

Quantization pipeline



Measurement tools

Value	Tool
Model latency, p50 and p95	`infer_bench` (our C++ harness), `CLOCK_MONOTONIC`, 200 runs after warm-up (YOLOv8n: 50)
CPU cycles, IPC, L2 traffic	`perf stat` 7.2, A53 PMU, user space; per frame = (N runs − 0 runs) / N
Peak RAM	`getrusage` `ru_maxrss` (separate run); app: `/proc/<pid>/status`
CPU load	`/proc/<pid>/stat` utime + stime per wall second (400 % = 4 cores)
GPU load and clock	DRM fdinfo `drm-engine-gpu` busy ns per wall second; clock from devfreq
GPU time per shader	ExecuTorch ETDump (Vulkan timestamp queries)
Temperature	hwmon (cpuss, gpu)
RPC to the STM32	`time.monotonic()` around each `set_motor` call, send to reply
Capture to motor command	Frame capture time → return of the RPC that starts the pulse
Live values	App `/state` endpoint, last 60 samples

LightFC precision

Processor, precision	p50 ms	p95 ms	Size MB	Peak RAM MB	CPU Mcycles/frame	L2 traffic MB/frame	OTB AUC	Max error vs PyTorch
CPU fp32	86.5	105.7	12.7	26	643	461	0.581	< 1e-5
CPU int8, in use	**49.6**	**77.5**	**3.4**	**13**	**358**	**31.7**	**0.625**	0.073
GPU fp32	364.5	365.8	12.7	70	3.4	3.5	–	8e-5
GPU fp16	325.9	327.0	6.4	70	9.8	4.6	0.581	0.004

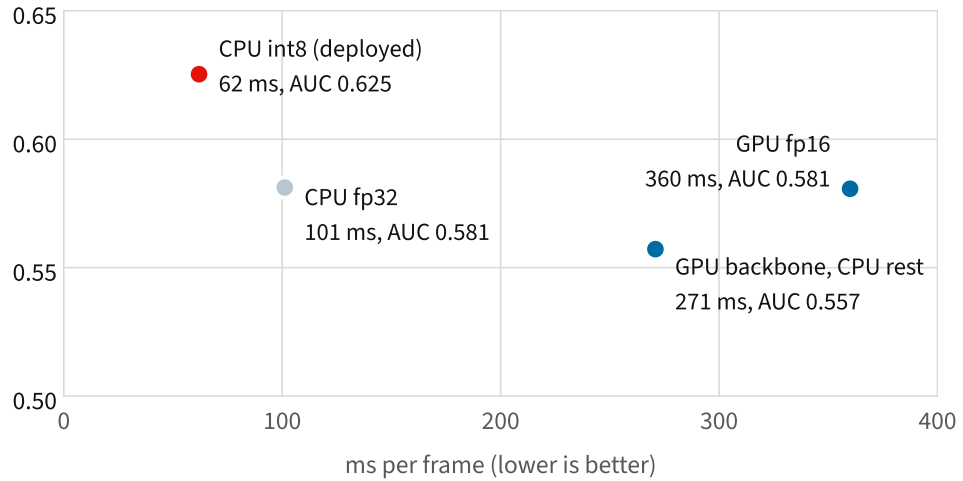
Backbone + fusion + head, 4 threads, 200 runs. AUC: 8 OTB100 test sequences, first 300 frames each (2,194 frames). Error: max score difference vs PyTorch, 20 crop pairs. –: not measured.

- CPU int8 vs fp32: −43 % latency, −73 % size, −50 % RAM, −93 % L2 traffic; AUC 0.625 vs 0.581.
- GPU fp16 vs fp32: −11 % latency, −50 % size; AUC unchanged.
- GPU vs CPU fp32: 4.2× slower (fp32), 3.8× slower (fp16).

Trackers compared on OTB100

int8: fastest, no AUC loss

LightFC, OTB success AUC (higher is better)



Source: SCORECARD.md, OTB 8 sequences, first 300 frames, app chain on the board

Tracker	Runtime	ms/frame	AUC	P@20
KCF	OpenCV	21.4	0.286	0.362
NanoTrack v2	OpenCV	28.5	0.544	0.786
LightFC fp32, authors	PyTorch, CUDA	–	0.616	0.857
LightFC fp32, ours	**ExecuTorch, CPU**	99.8	**0.620**	**0.856**

Full sequences, 5,709 frames. ms/frame: one update on the board, crop included. Authors: their published trajectories, scored with our script.

NanoTrack: 3.5× faster, AUC 0.075 lower. Our port vs the authors' boxes: mean IoU 0.938.

YOLOv8n precision

Processor, precision	p50 ms	p95 ms	Size MB	Peak RAM MB	CPU Mcycles/frame	L2 traffic MB/frame	Box IoU vs CPU fp32	Use
CPU fp32	111.9	124.8	12.7	32	709	157	1.000	–
CPU int8	**89.4**	**93.0**	**3.3**	**23**	554	66.5	0.972	**SEARCH**
GPU fp32	789.4	790.5	12.7	72	6.2	6.0	1.000	–
GPU fp16	**615.7**	616.8	6.4	72	**5.6**	**5.9**	0.999	**TRACK check**

4 threads, 50 runs. Box IoU vs CPU fp32 on the first frame of each OTB100 test sequence. CPU fp32 vs Ultralytics: 0.994. No mAP.

- CPU int8 vs fp32: −20 % latency, −74 % size, −29 % RAM, −58 % L2 traffic.
- int8 scores saturate near 0.80; with several people, the top box can change.
- GPU fp16 vs fp32: −22 % latency; boxes unchanged.
- int8 holds because the network outputs raw values; box decode runs in fp32 (NumPy).

Efficiency scorecard

Metric	LightFC portable fp32	LightFC XNNPACK fp32	LightFC XNNPACK int8	YOLOv8n XNNPACK fp32	YOLOv8n XNNPACK int8
Model size, MB	18.2	12.7	**3.4**	12.7	**3.3**
Peak RAM, MB	23	26	**13**	32	**23**
Latency p50, ms	34,734	86.5	**49.6**	111.9	**89.4**
Throughput, FPS	0.03	11.6	**20.2**	8.9	**11.2**
Energy	Not measured				
CPU cycles per frame, M	69,511	643	**358**	709	**554**
L2 traffic per frame, MB	3,626	461	**31.7**	157	**66.5**

Portable fp32: ExecuTorch reference kernels, no XNNPACK. The XNNPACK LightFC columns use the authors' inference form of the model. Int8 columns are the models in use. No power sensor on the board; CPU cycles and L2 traffic (A53 PMU) stand in for energy.

Live system

Metric	p50	p95
Tracking rate (TRACK)	14.4 FPS	
Capture to motor command	142 ms	183 ms
Tracker step (crop + LightFC int8 + decode)	67 ms	77 ms
Detector step (SEARCH, CPU int8)	131 ms	163 ms
GPU check (fp16)	648 ms	661 ms
RPC to the STM32	6.3 ms	9.1 ms
Application RAM	200 MB	

- Camera: 15.0 FPS.
- 1,451 frames: 0 exceptions, 0 RPC errors. 17 of 73 GPU checks discarded: the camera moved after capture.

Source: app /state endpoint, last 60 samples, 2026-10-13, 4 XNNPACK threads, one person walking.

Lessons

- int8 pays off on the A53: −43 % (LightFC), −20 % (YOLOv8n).
- Quantize the network, not the box decode.
- Per-frame work on the CPU, periodic work on the GPU.
- A similarity tracker drifts onto background; a 1 Hz detector check catches it.
- Motion blur corrupts the position error; measure on still frames only.

Future work

1. Smoother motor motion while tracking.
2. YOLOv8n with a single-class output.
3. Tracking tests on smaller targets, for precision.

Questions